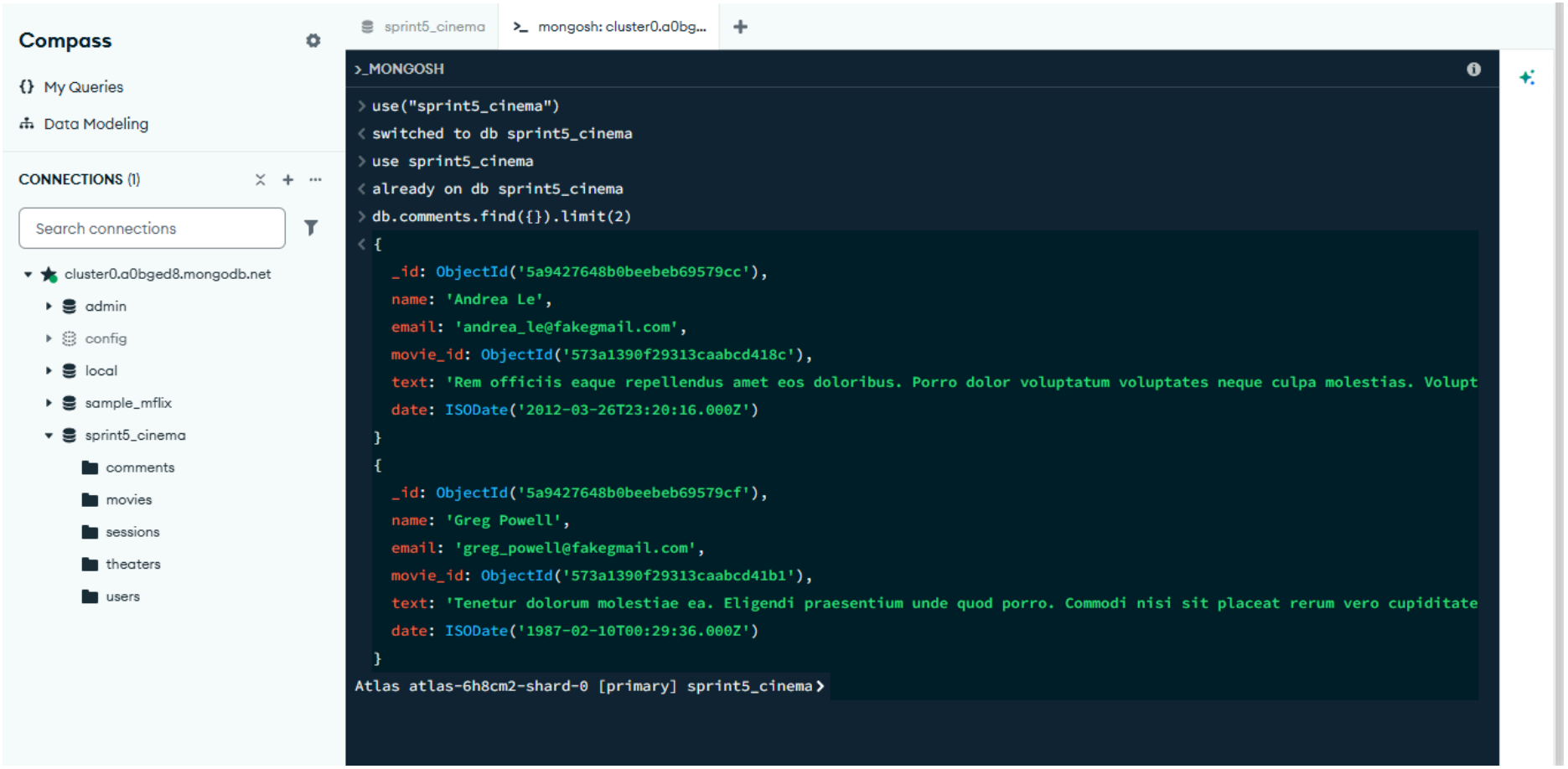
Nivel 1

Nivel 1 - Ejercicio 1A: Mostrar los dos primeros comentarios

Para resolver este apartado utilicé el método `find({})` sobre la colección `comments`, que permite consultar los documentos almacenados en MongoDB. Dejé las llaves vacías porque el ejercicio no requiere aplicar ninguna condición de filtrado.

Después añadí el método `limit(2)` para restringir el resultado a los dos primeros documentos devueltos por la consulta, evitando recuperar innecesariamente todos los comentarios de la colección.

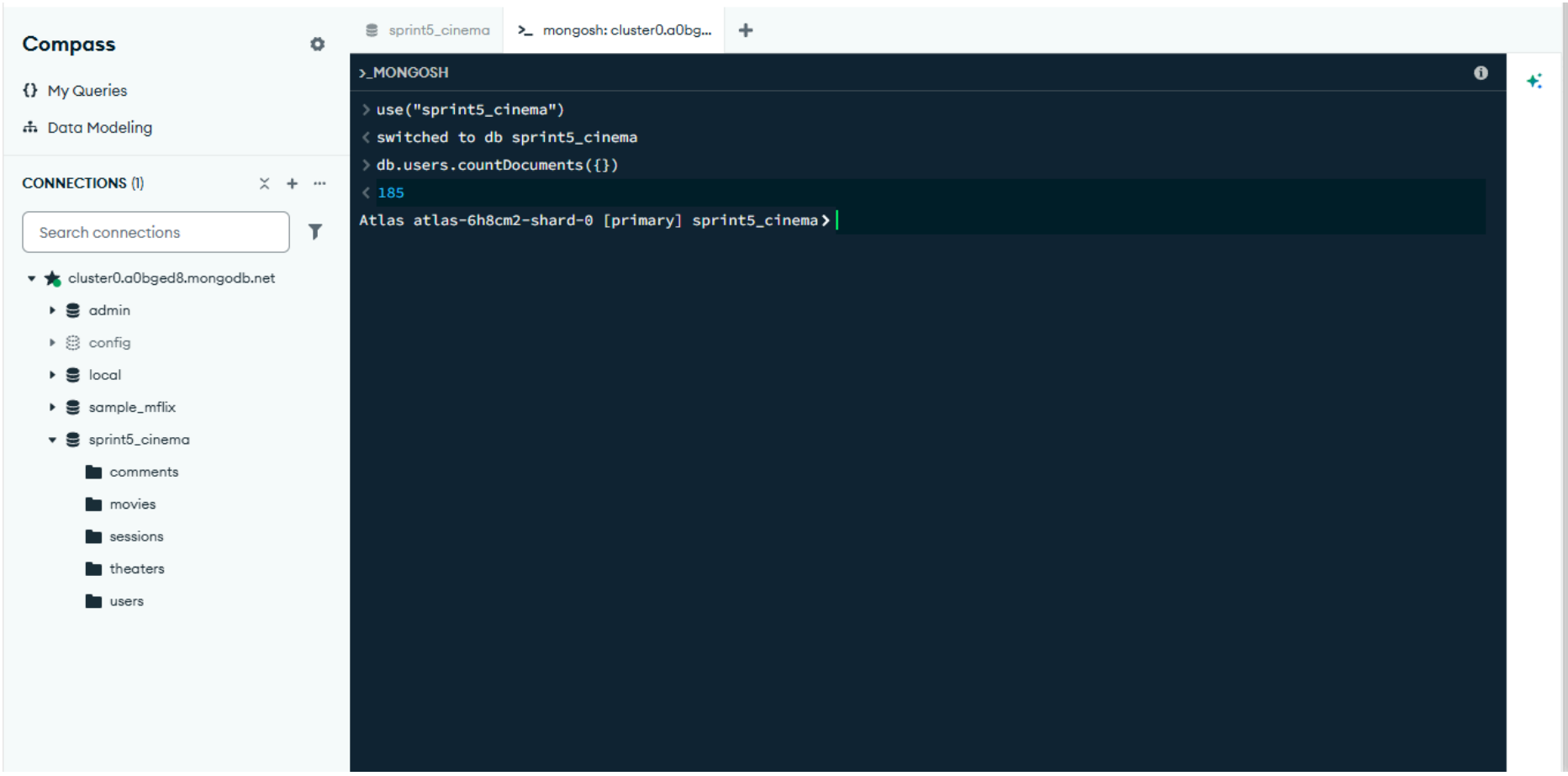
Como resultado, obtuve dos documentos con sus respectivos campos, como el nombre del usuario, correo electrónico, identificador de la película, texto y fecha del comentario.


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
cluster0.a0bged8.mongodb.net
admin
config
local
sample_mflix
sprint5_cinema
comments
movies
sessions
theaters
users
sprint5_cinema
mongosh: cluster0.a0bg...
>_MONGOSH
> use("sprint5_cinema")
< switched to db sprint5_cinema
> use sprint5_cinema
< already on db sprint5_cinema
> db.comments.find({}).limit(2)
< {
_id: ObjectId('5a9427648b0beebeb69579cc'),
name: 'Andrea Le',
email: 'andrea_le@fakegmail.com',
movie_id: ObjectId('573a1390f29313caabcd418c'),
text: 'Rem officiis eaque repellendus amet eos doloribus. Porro dolor voluptatum voluptates neque culpa molestias. Volupt
date: ISODate('2012-03-26T23:20:16.000Z')
}
{
_id: ObjectId('5a9427648b0beebeb69579cf'),
name: 'Greg Powell',
email: 'greg_powell@fakegmail.com',
movie_id: ObjectId('573a1390f29313caabcd41b1'),
text: 'Tenetur dolorum molestiae ea. Eligendi praesentium unde quod porro. Commodi nisi sit placeat rerum vero cupiditate
date: ISODate('1987-02-10T00:29:36.000Z')
}
Atlas atlas-6h8cm2-shard-0 [primary] sprint5_cinema>

Nivel 1 - Ejercicio 1B: Total de usuarios registrados

Para conocer la cantidad de usuarios registrados utilicé el método `countDocuments({})` sobre la colección `users`. Este método permite contar los documentos que cumplen una condición. En este caso, dejé las llaves vacías porque necesitaba contar todos los usuarios, sin aplicar ningún filtro.

La consulta devolvió un total de **185 usuarios registrados**.

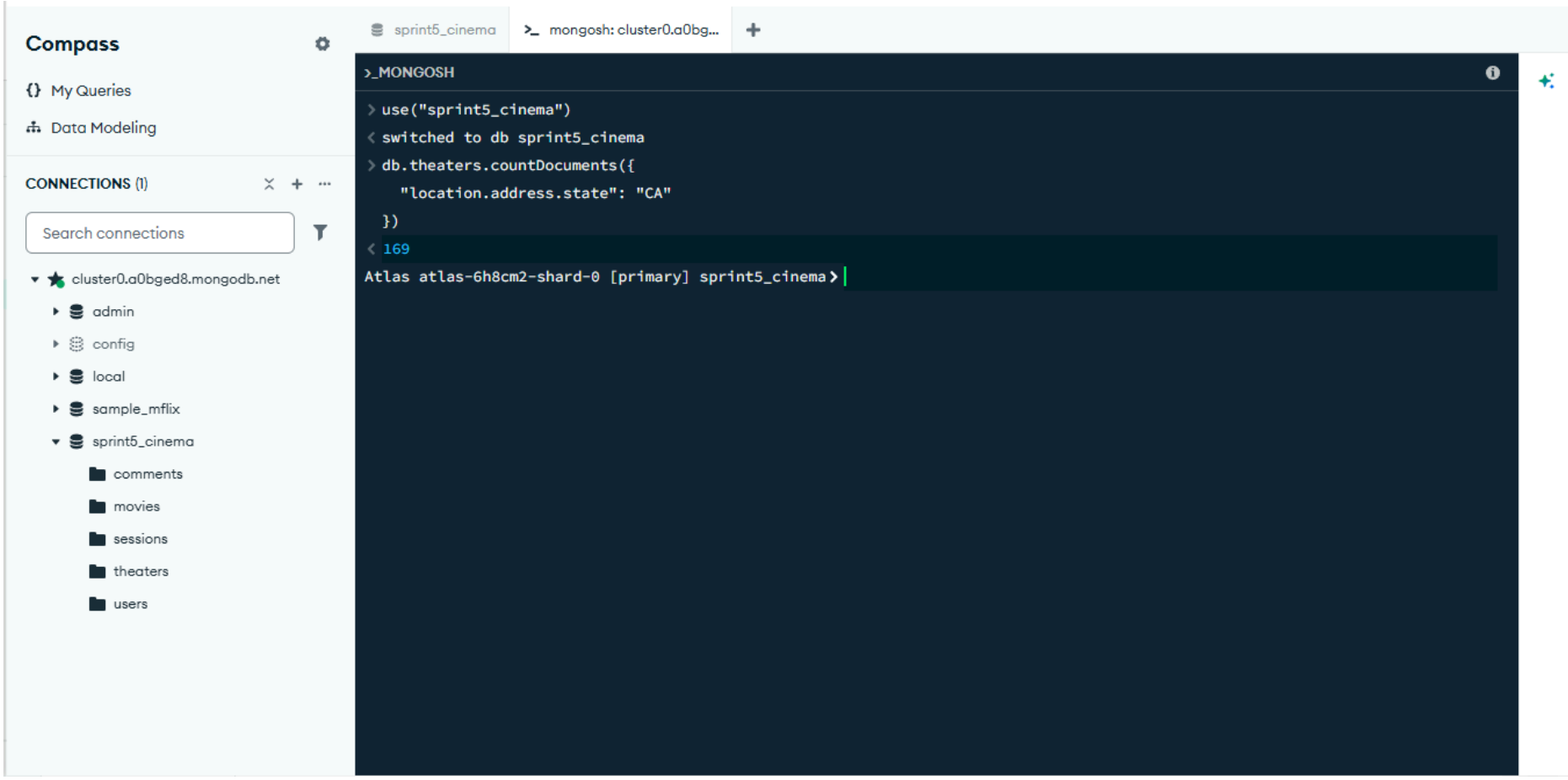

Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
cluster0.a0bged8.mongodb.net
admin
config
local
sample_mflix
sprint5_cinema
comments
movies
sessions
theaters
users
sprint5_cinema
mongosh: cluster0.a0bg...
>_MONGOSH
use("sprint5_cinema")
switched to db sprint5_cinema
db.users.countDocuments({})
185
Atlas atlas-6h8cm2-shard-0 [primary] sprint5_cinema>

Nivel 1 - Ejercicio 1C: Cines en California

Para obtener la cantidad de cines ubicados en California utilicé el método `countDocuments()` sobre la colección `theaters`.

Dentro del filtro especifiqué el campo `"location.address.state"`, utilizando la notación de puntos para acceder a un campo anidado dentro del documento. Establecí la condición `"CA"` porque es el código que identifica al estado de California.

De esta manera, MongoDB cuenta únicamente los documentos que cumplen esa condición, obteniendo un resultado de **169 cines**.


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
cluster0.a0bged8.mongodb.net
admin
config
local
sample_mflix
sprint5_cinema
comments
movies
sessions
theaters
users
sprint5_cinema
mongosh: cluster0.a0bg...
>_MONGOSH
> use("sprint5_cinema")
< switched to db sprint5_cinema
> db.theaters.countDocuments({
"location.address.state": "CA"
})
< 169
Atlas atlas-6h8cm2-shard-0 [primary] sprint5_cinema>

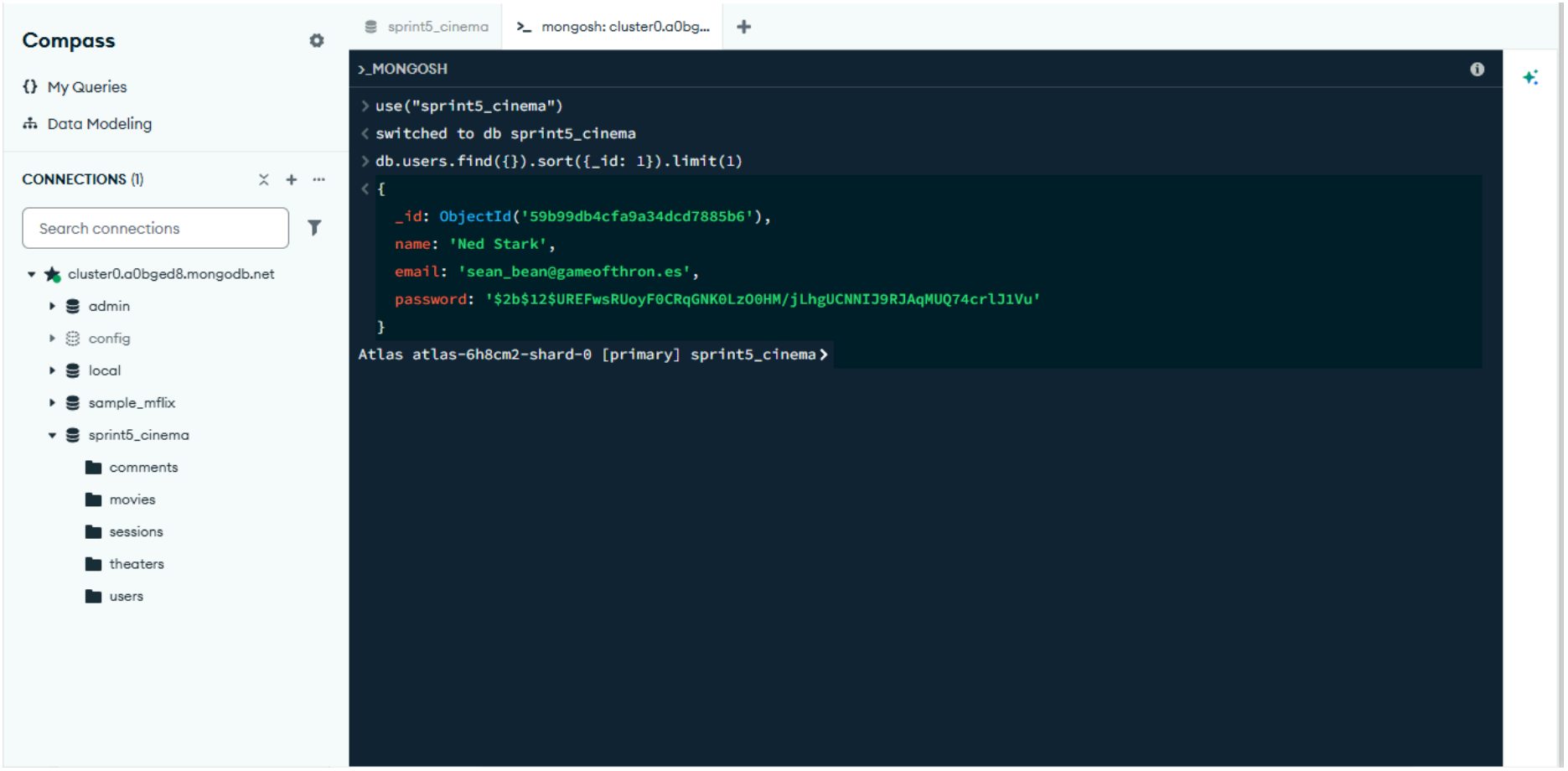
Nivel 1 - Ejercicio 1D: Primer usuario registrado

Para identificar al primer usuario registrado utilicé el método `find({})` sobre la colección `users`, sin aplicar filtros, ya que necesitaba consultar todos los documentos.

Después incorporé `sort({_id: 1})` para ordenar los usuarios de forma ascendente según su identificador `_id`. Como estos identificadores son de tipo `ObjectId`, contienen una marca temporal que permite aproximarnos al orden de creación de los registros.

Finalmente, utilicé `limit(1)` para obtener únicamente el primer documento de la lista ordenada, evitando mostrar los demás usuarios.

De esta manera, pude identificar al usuario con el identificador más antiguo de la colección, utilizándolo como referencia para determinar quién se registró primero.

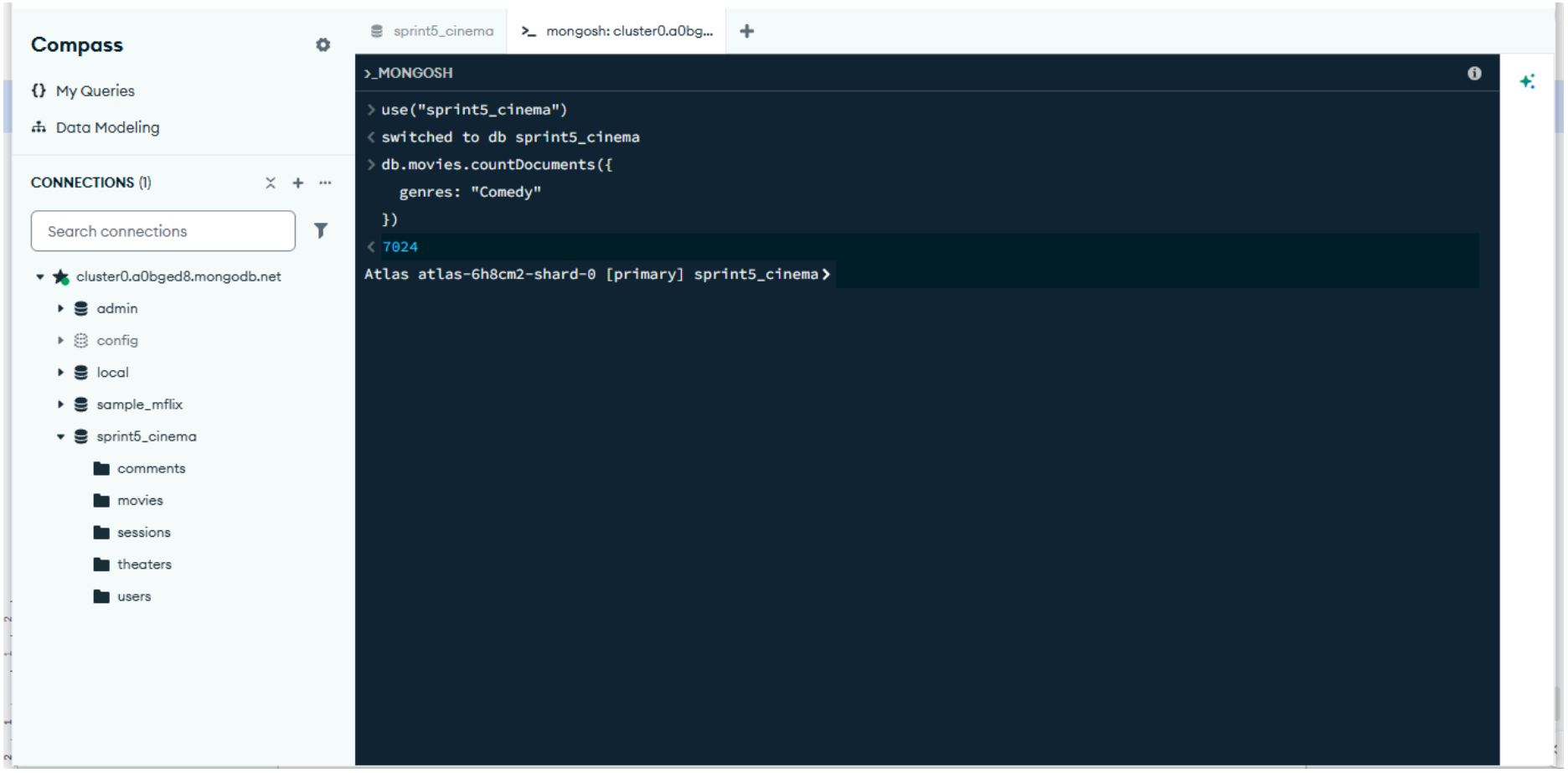

Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
cluster0.a0bged8.mongodb.net
admin
config
local
sample_mflix
sprint5_cinema
comments
movies
sessions
theaters
users
sprint5_cinema
mongosh: cluster0.a0bg...
>_MONGOSH
use("sprint5_cinema")
switched to db sprint5_cinema
db.users.find({}).sort({_id: 1}).limit(1)
{
_id: ObjectId('59b99db4cfa9a34dcd7885b6'),
name: 'Ned Stark',
email: 'sean_bean@gameofthron.es',
password: '$2b$12$UREFwsRUoyF0CRqGNK0LzO0HM/jLhgUCNNIJ9RJAqMUQ74crlJ1Vu'
}
Atlas atlas-6h8cm2-shard-0 [primary] sprint5_cinema>

Nivel 1 - Ejercicio 1E: Cantidad de películas de comedia

Para conocer cuántas películas de comedia hay en la base de datos, utilicé el método `countDocuments()` sobre la colección `movies`, aplicando un filtro sobre el campo `genres`.

Dentro del filtro establecí la condición `genres: "Comedy"` para seleccionar únicamente las películas que contienen el género comedia. Como este campo almacena varios géneros en un array, MongoDB permite buscar directamente un valor dentro de él, sin necesidad de recorrer sus elementos individualmente.

Finalmente, `countDocuments()` devuelve la cantidad total de documentos que cumplen esta condición, incluyendo las películas que pertenecen al género comedia aunque también estén clasificadas en otros géneros.


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
cluster0.a0bged8.mongodb.net
admin
config
local
sample_mflix
sprint5_cinema
comments
movies
sessions
theaters
users
sprint5_cinema
mongosh: cluster0.a0bg...
>_MONGOSH
> use("sprint5_cinema")
< switched to db sprint5_cinema
> db.movies.countDocuments({
genres: "Comedy"
})
< 7024
Atlas atlas-6h8cm2-shard-0 [primary] sprint5_cinema>

Nivel 1 - Ejercicio 2: Películas de 1932 del género drama o en francés

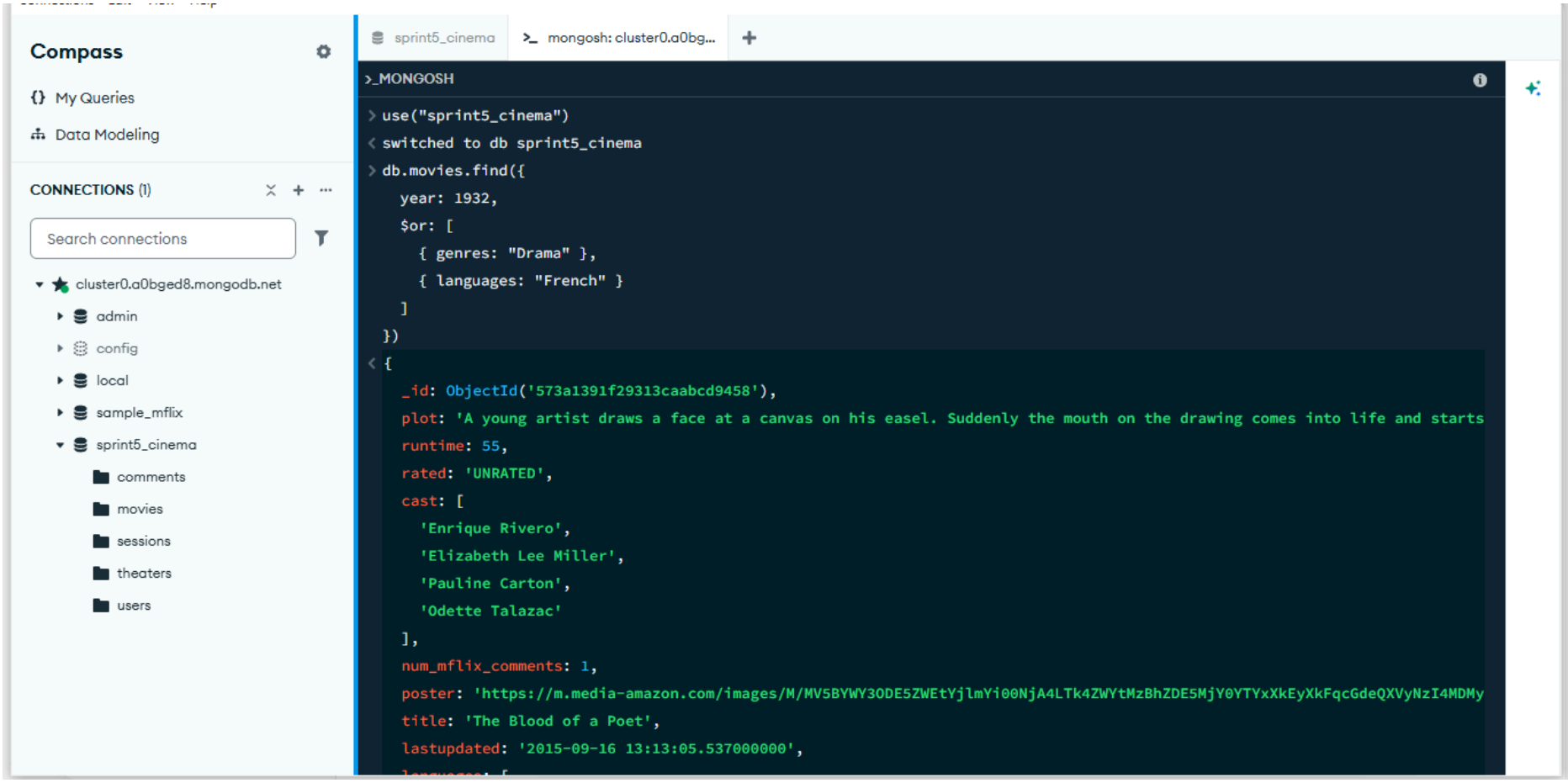
Para resolver este ejercicio utilicé el método `find()` sobre la colección `movies`, estableciendo diferentes condiciones de filtrado.

Primero utilicé `year: 1932` para seleccionar únicamente las películas producidas durante ese año. Después incorporé el operador lógico `$or`, que permite recuperar documentos que cumplan al menos una de las condiciones indicadas.

Dentro de `$or` establecí dos filtros: `genres: "Drama"` para seleccionar las películas de género dramático y `languages: "French"` para incluir las películas que tengan el francés entre sus idiomas.

Al combinar el año con el operador `$or` dentro de un mismo objeto, MongoDB aplica implícitamente una condición `AND`. De esta manera, los documentos obtenidos corresponden exclusivamente a películas de 1932 que cumplen al menos una de las dos condiciones solicitadas.

Finalmente, utilicé `find()` sin limitar ni proyectar campos, ya que el enunciado solicita mostrar los documentos completos de las películas que cumplen los criterios.


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
cluster0.a0bged8.mongodb.net
admin
config
local
sample_mflix
sprint5_cinema
comments
movies
sessions
theaters
users
sprint5_cinema
mongosh: cluster0.a0bg...
>_MONGOSH
> use("sprint5_cinema")
< switched to db sprint5_cinema
> db.movies.find({
year: 1932,
$or: [
{ genres: "Drama" },
{ languages: "French" }
]
})
< {
_id: ObjectId('573a1391f29313caabcd9458'),
plot: 'A young artist draws a face at a canvas on his easel. Suddenly the mouth on the drawing comes into life and starts
runtime: 55,
rated: 'UNRATED',
cast: [
'Enrique Rivero',
'Elizabeth Lee Miller',
'Pauline Carton',
'Odette Talazac'
],
num_mflix_comments: 1,
poster: 'https://m.media-amazon.com/images/M/MV5BYWY3ODE5ZWEtYjlmYi00NjA4LTk4ZWYtMzBhZDE5MjY0YTYxXkEyXkFqcGdeQXVyNzI4MDMy
title: 'The Blood of a Poet',
lastupdated: '2015-09-16 13:13:05.537000000',

Nivel 1 - Ejercicio 3: Películas estadounidenses con entre 5 y 9 premios producidas entre 2012 y 2014

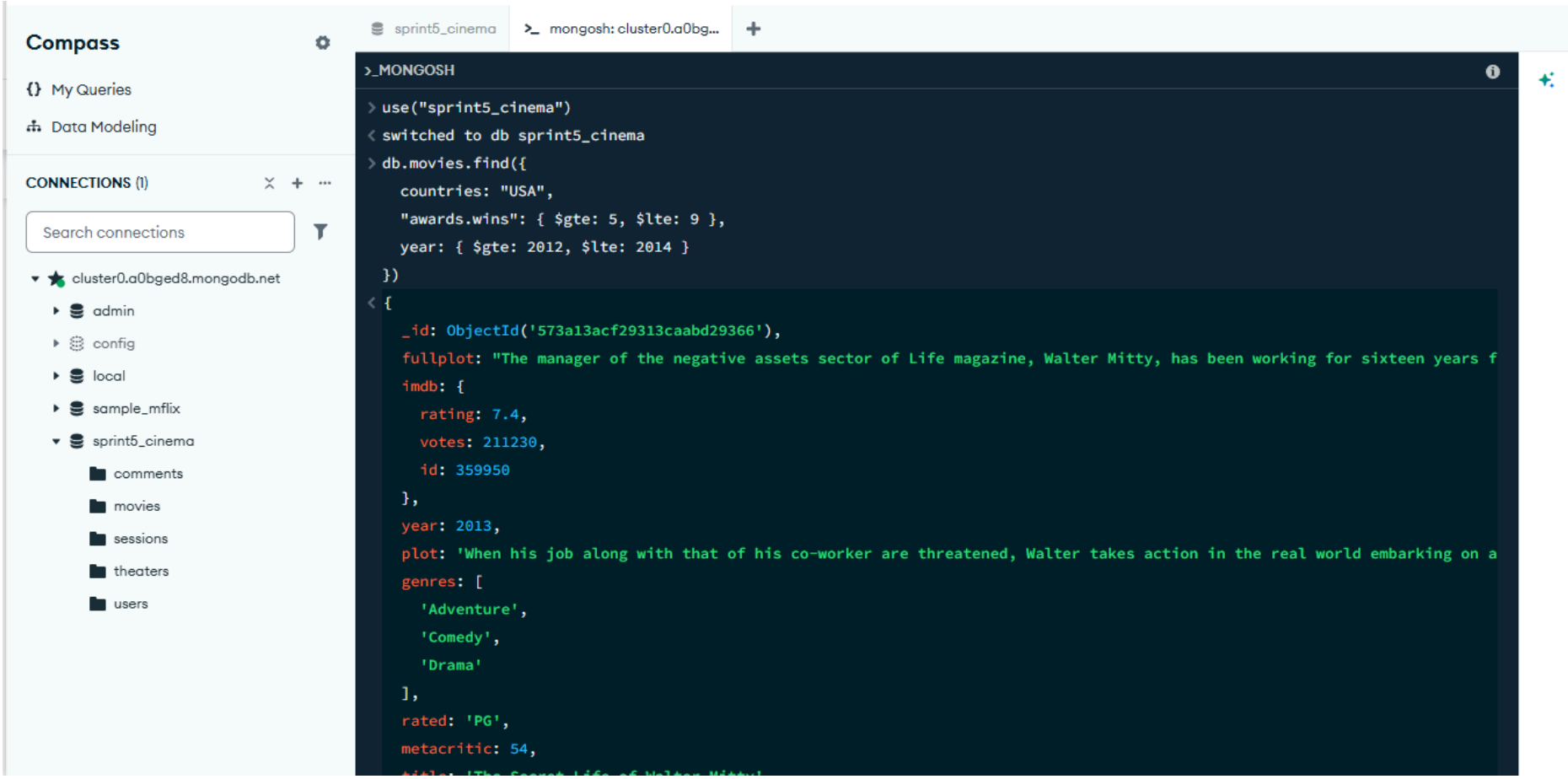
Para resolver este ejercicio utilicé el método `find()` sobre la colección `movies`, combinando tres condiciones de filtrado.

Primero establecí `countries: "USA"` para seleccionar las películas producidas en Estados Unidos. Después utilicé el campo anidado `"awards.wins"` para consultar la cantidad de premios ganados, aplicando los operadores `$gte` y `$lte` para establecer un rango inclusivo de entre 5 y 9 premios.

A continuación, apliqué estos mismos operadores sobre el campo `year` para seleccionar únicamente las películas producidas entre 2012 y 2014.

No fue necesario utilizar explícitamente `$and`, ya que MongoDB interpreta las condiciones del mismo objeto como filtros que deben cumplirse simultáneamente.

Finalmente, utilicé `find()` sin limitar los resultados ni excluir campos, porque el ejercicio solicita mostrar todos los documentos que cumplen las condiciones establecidas.


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
cluster0.a0bged8.mongodb.net
admin
config
local
sample_mflix
sprint5_cinema
comments
movies
sessions
theaters
users
sprint5_cinema
mongosh: cluster0.a0bg...
>_MONGOSH
use("sprint5_cinema")
switched to db sprint5_cinema
db.movies.find({
countries: "USA",
"awards.wins": { $gte: 5, $lte: 9 },
year: { $gte: 2012, $lte: 2014 }
})
{
_id: ObjectId('573a13acf29313caabd29366'),
fullplot: "The manager of the negative assets sector of Life magazine, Walter Mitty, has been working for sixteen years f
imdb: {
rating: 7.4,
votes: 211230,
id: 359950
},
year: 2013,
plot: 'When his job along with that of his co-worker are threatened, Walter takes action in the real world embarking on a
genres: [
'Adventure',
'Comedy',
'Drama'
],
rated: 'PG',
metacritic: 54,

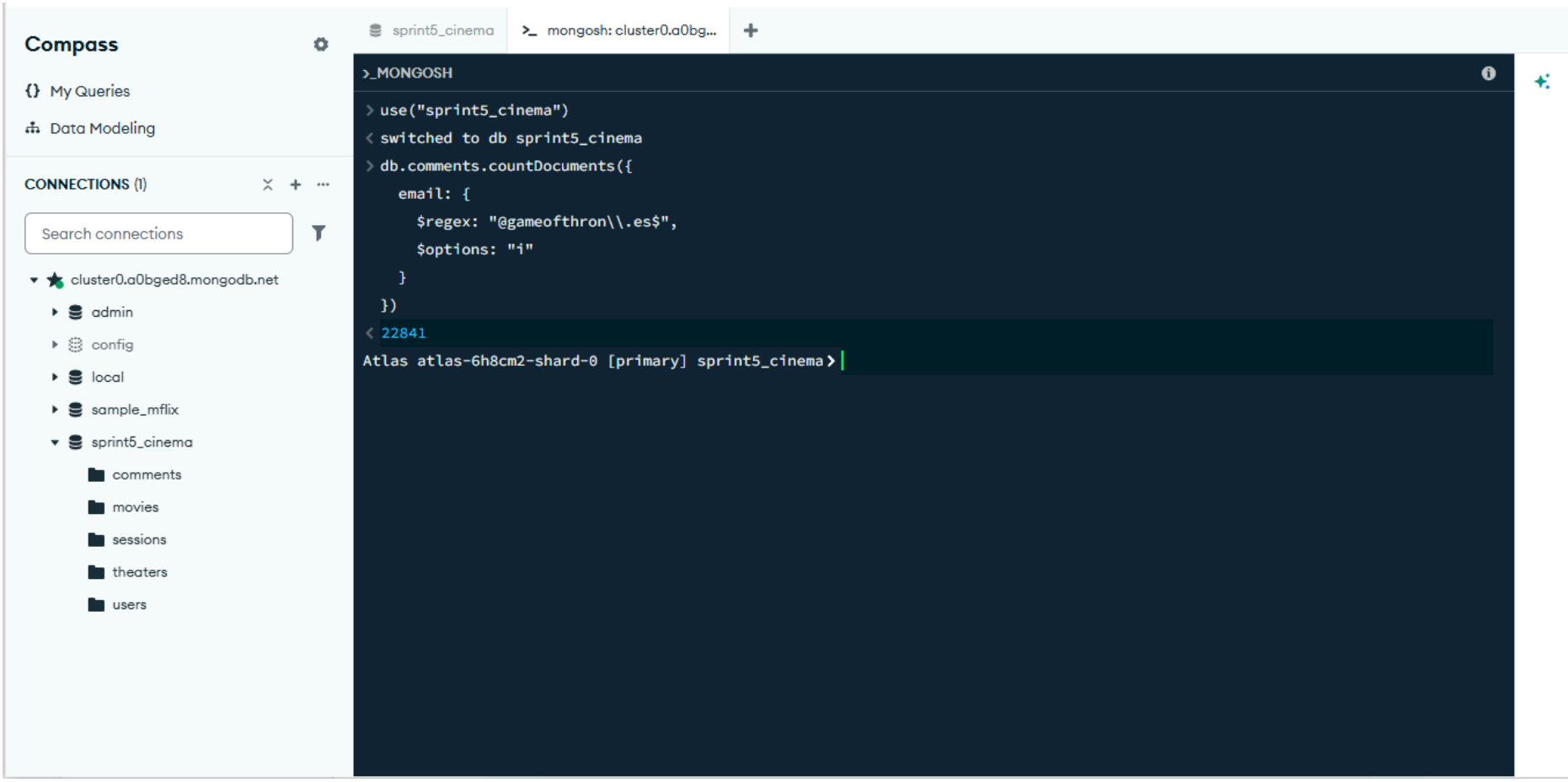
Nivel 2 - Ejercicio 1: Comentarios de usuarios con dominio GAMEOFTHRON.ES

Para resolver este ejercicio utilicé el método `countDocuments()` sobre la colección `comments`, aplicando un filtro al campo `email` para contar únicamente los comentarios escritos por usuarios con el dominio `gameofthron.es`.

Para establecer esta condición utilicé el operador `$regex`, que permite buscar coincidencias mediante patrones de texto. Especifiqué el dominio `@gameofthron.es` y añadí el símbolo `$` al final para asegurar que las direcciones de correo terminen exactamente con ese dominio.

También incorporé `$options: "i"` para que la búsqueda no distinga entre mayúsculas y minúsculas.

Finalmente, `countDocuments()` devuelve la cantidad total de comentarios que cumplen esta condición, sin necesidad de recuperar todos los documentos de la colección.


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
cluster0.a0bged8.mongodb.net
admin
config
local
sample_mflix
sprint5_cinema
comments
movies
sessions
theaters
users
sprint5_cinema
mongosh: cluster0.a0bg...
>_MONGOSH
> use("sprint5_cinema")
< switched to db sprint5_cinema
> db.comments.countDocuments({
email: {
$regex: "@gameofthron\\.es$",
$options: "i"
}
})
< 22841
Atlas atlas-6h8cm2-shard-0 [primary] sprint5_cinema>

Nivel 2 - Ejercicio 2: Cantidad de cines por código postal en Washington D. C.

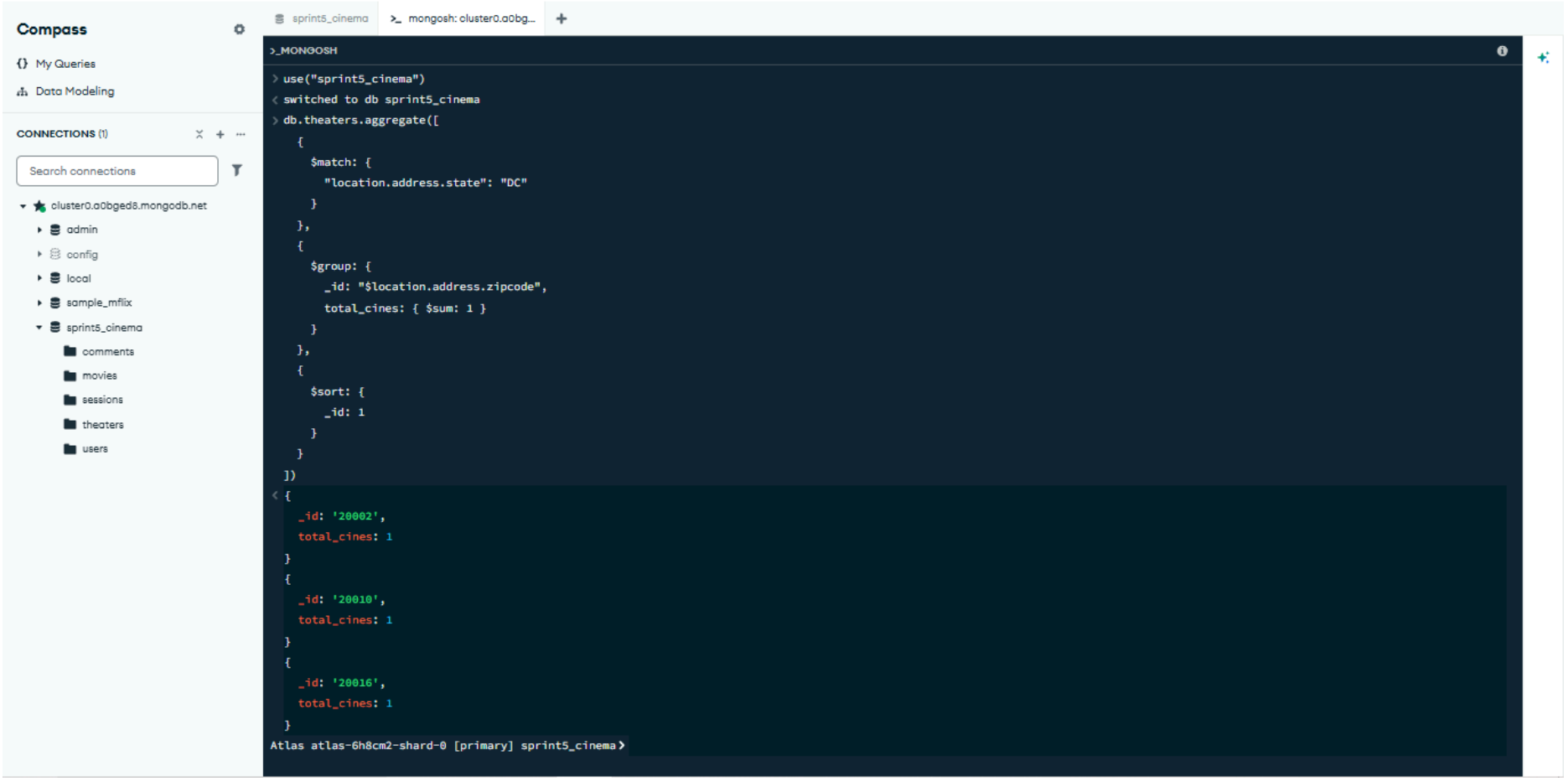
Para resolver este ejercicio utilicé el método `aggregate()` sobre la colección `theaters`, construyendo una consulta mediante tres etapas.

Primero apliqué `$match` para filtrar los documentos cuyo campo `"location.address.state"` contiene el valor `"DC"`, seleccionando únicamente los cines ubicados en Washington D. C.

Después utilicé `$group` para agrupar los documentos según el campo `"location.address.zipcode"`. Dentro de esta etapa definí `_id` como el código postal y utilicé `$sum: 1` para contar la cantidad de cines correspondientes a cada grupo, guardando el resultado en el campo `total_cines`.

Finalmente, incorporé `$sort` con el valor `1` sobre `_id` para ordenar los códigos postales de forma ascendente.

De esta manera, obtuve una relación de los diferentes códigos postales de Washington D. C. junto con la cantidad de cines registrados en cada uno.


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
cluster0.a0bged8.mongodb.net
admin
config
local
sample_mflix
sprint5_cinema
comments
movies
sessions
theaters
users
sprint5_cinema
mongosh: cluster0.a0bg...
>_MONGOSH
> use("sprint5_cinema")
< switched to db sprint5_cinema
> db.theaters.aggregate([
{
$match: {
"location.address.state": "DC"
}
},
{
$group: {
_id: "$location.address.zipcode",
total_cines: { $sum: 1 }
}
},
{
$sort: {
_id: 1
}
}
])
< {
_id: '20002',
total_cines: 1
}
{
_id: '20010',
total_cines: 1
}
{
_id: '20016',
total_cines: 1
}
Atlas atlas-6h8cm2-shard-0 [primary] sprint5_cinema>

Nivel 3 - Ejercicio 1: Películas de John Landis con puntuación IMDb entre 7,5 y 8

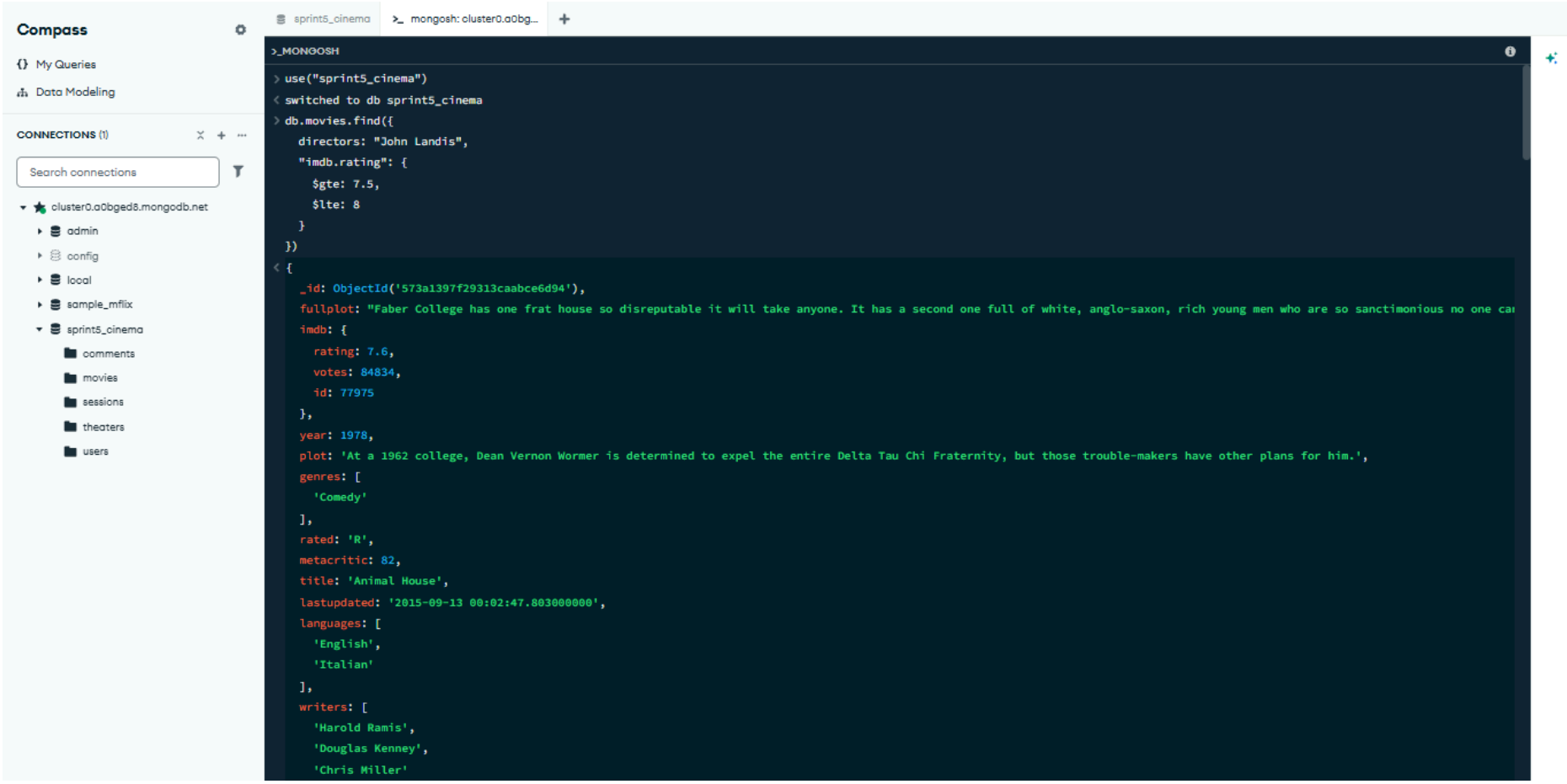
Para resolver este ejercicio utilicé el método `find()` sobre la colección `movies`, aplicando dos condiciones de filtrado.

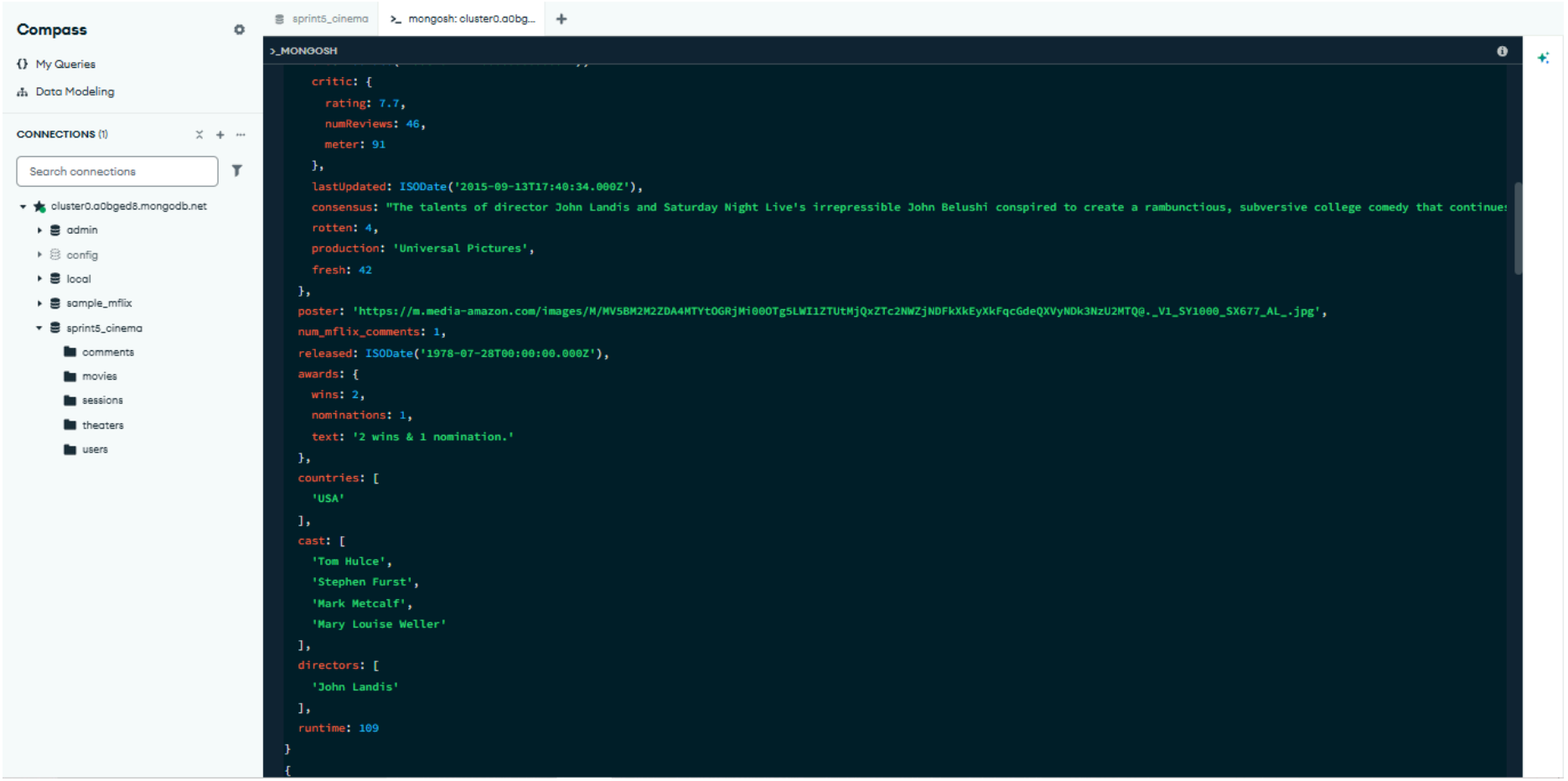
Primero establecí `directors: "John Landis"` para seleccionar las películas que incluyen a este director. Como el campo `directors` almacena los nombres en un array, MongoDB permite buscar directamente el valor indicado sin necesidad de recorrer sus elementos.

Después utilicé `"imdb.rating"` para acceder al campo anidado que contiene la puntuación IMDb de cada película. Sobre este campo apliqué los operadores `$gte` y `$lte`, estableciendo un rango inclusivo entre 7,5 y 8.

Ambas condiciones se combinan mediante un `AND` implícito, por lo que los documentos recuperados deben cumplir los dos criterios simultáneamente.

Finalmente, mantuve `find()` sin limitar los resultados para mostrar todos los documentos que cumplen las condiciones solicitadas.


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
cluster0.a0bged8.mongodb.net
admin
config
local
sample_mflix
sprint5_cinema
comments
movies
sessions
theaters
users
sprint5_cinema
mongosh: cluster0.a0bg...
>_MONGOSH
use("sprint5_cinema")
switched to db sprint5_cinema
db.movies.find({
directors: "John Landis",
"imdb.rating": {
$gte: 7.5,
$lte: 8
}
})
{
_id: ObjectId('573a1397f29313caabce6d94'),
fullplot: "Faber College has one frat house so disreputable it will take anyone. It has a second one full of white, anglo-saxon, rich young men who are so sanctimonious no one ca
imdb: {
rating: 7.6,
votes: 84834,
id: 77975
},
year: 1978,
plot: 'At a 1962 college, Dean Vernon Wormer is determined to expel the entire Delta Tau Chi Fraternity, but those trouble-makers have other plans for him.',
genres: [
'Comedy'
],
rated: 'R',
metacritic: 82,
title: 'Animal House',
lastupdated: '2015-09-13 00:02:47.803000000',
languages: [
'English',
'Italian'
],
writers: [
'Harold Ramis',
'Douglas Kenney',
'Chris Miller'


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
cluster0.a0bged8.mongodb.net
admin
config
local
sample_mflix
sprint5_cinema
comments
movies
sessions
theaters
users
sprint5_cinema
mongosh: cluster0.a0bg...
>_MONGOSH
critic: {
rating: 7.7,
numReviews: 46,
meter: 91
},
lastUpdated: ISODate('2015-09-13T17:40:34.000Z'),
consensus: "The talents of director John Landis and Saturday Night Live's irrepressible John Belushi conspired to create a rambunctious, subversive college comedy that continue
rotten: 4,
production: 'Universal Pictures',
fresh: 42
},
poster: 'https://m.media-amazon.com/images/M/MV5BM2M2ZDA4MTYtOGRjMi00OTg5LWI1ZTUtMjQxZTc2NWZjNDFkXkEyXkFqcGdeQXVyNDk3NzU2MTQ@._V1_SY1000_SX677_AL_.jpg',
num_mflix_comments: 1,
released: ISODate('1978-07-28T00:00:00.000Z'),
awards: {
wins: 2,
nominations: 1,
text: '2 wins & 1 nomination.'
},
countries: [
'USA'
],
cast: [
'Tom Hulce',
'Stephen Furst',
'Mark Metcalf',
'Mary Louise Weller'
],
directors: [
'John Landis'
],
runtime: 109
}
{

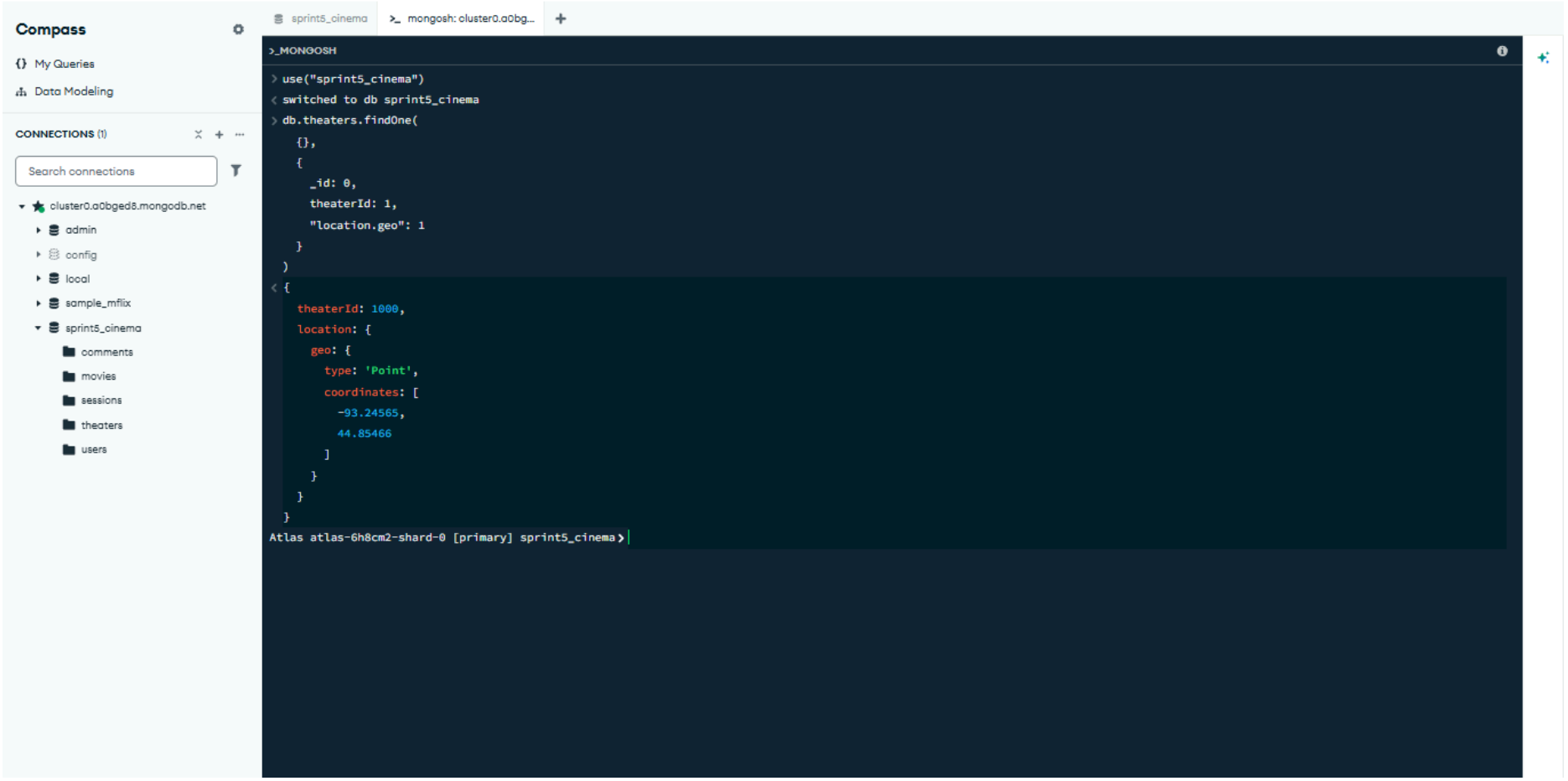
Nivel 3 - Ejercicio 2: Representación geográfica de los cines

Para resolver este ejercicio trabajé con la colección `theaters`, que contiene la información de los cines y sus ubicaciones geográficas.

Primero utilicé `findOne()` para examinar la estructura de los documentos e identificar el campo `location.geo`, donde se almacenan las coordenadas geográficas mediante el formato GeoJSON.

Utilicé una proyección para mostrar únicamente el identificador del cine y su ubicación, facilitando la comprobación de los datos necesarios para construir el mapa.

Este formato permite representar las ubicaciones mediante pares de longitud y latitud, que posteriormente se utilizan para visualizar la distribución geográfica de los cines.


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
cluster0.a0bged8.mongodb.net
admin
config
local
sample_mflix
sprint5_cinema
comments
movies
sessions
theaters
users
sprint5_cinema
mongosh: cluster0.a0bg...
>_MONGOSH
use("sprint5_cinema")
switched to db sprint5_cinema
db.theaters.findOne(
{},
{
_id: 0,
theaterId: 1,
"location.geo": 1
}
)
{
theaterId: 1000,
location: {
geo: {
type: 'Point',
coordinates: [
-93.24565,
44.85466
]
}
}
}
Atlas atlas-6h8cm2-shard-0 [primary] sprint5_cinema>

Nivel 3 - Ejercicio 2: Representación geográfica de los cines

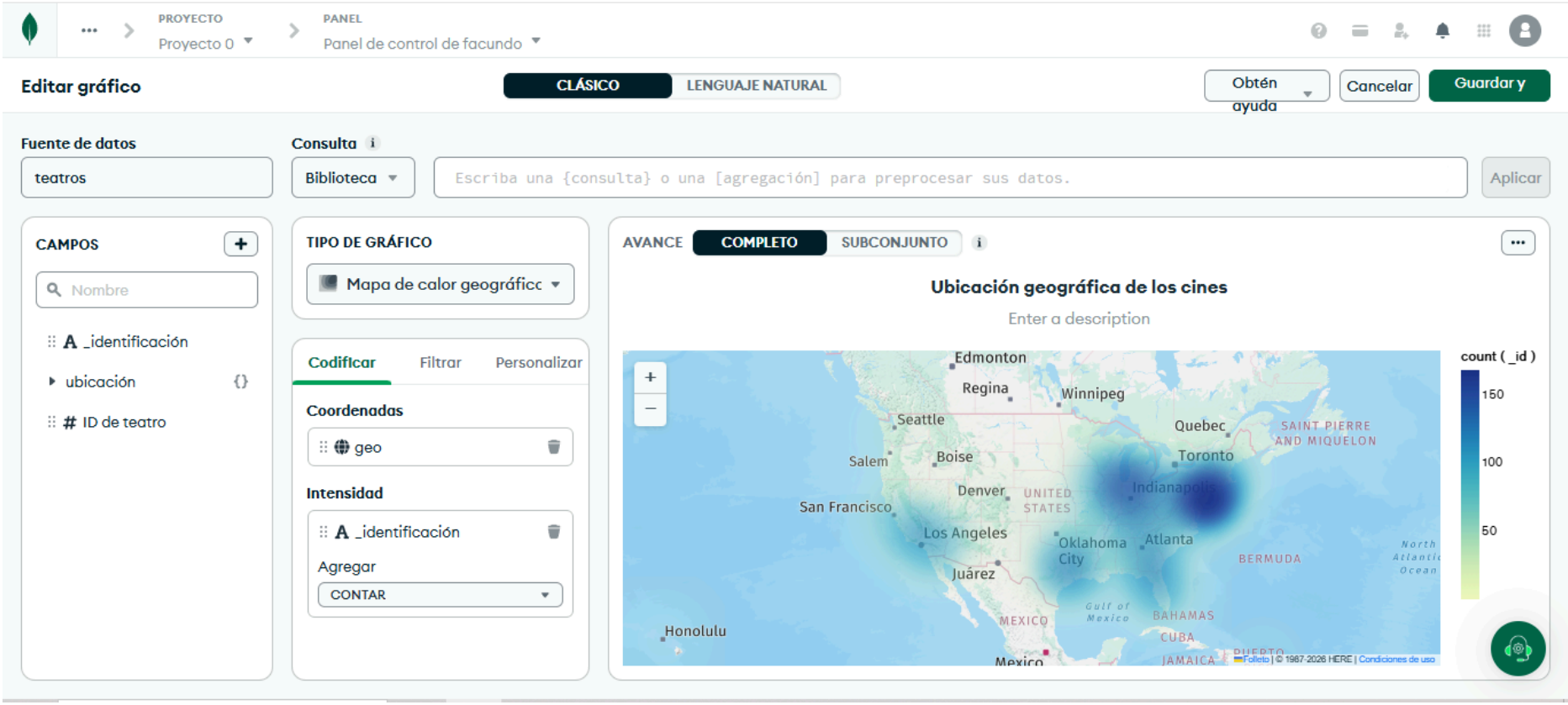
Para resolver este ejercicio utilicé la colección `theaters`, que contiene 1.564 documentos con información sobre los cines y sus ubicaciones.

Primero comprobé mediante `findOne()` que las coordenadas estaban almacenadas en el campo `location.geo` utilizando el formato GeoJSON. Después utilicé `countDocuments()` para verificar que los 1.564 documentos tenían coordenadas numéricas disponibles.

A continuación, utilicé MongoDB Atlas Charts para crear un mapa de calor geográfico. Configuré el campo `geo` como coordenadas y apliqué la agregación `COUNT` sobre `_id` para representar la concentración de cines en las distintas zonas.

Elegí este tipo de gráfico porque permite visualizar la distribución geográfica de los establecimientos y distinguir fácilmente las áreas con mayor concentración de cines.

El resultado es un mapa interactivo que representa la distribución espacial de los cines registrados en nuestra base de datos.


PROYECTO
Proyecto 0
PANEL
Panel de control de facundo
Editar gráfico
CLÁSICO
LENGUAJE NATURAL
Obtén ayuda
Cancelar
Guardar y
Fuente de datos
teatros
Consulta
Biblioteca
Escriba una {consulta} o una [agregación] para preprocesar sus datos.
Aplicar
CAMPOS
Nombre
A _identificación
ubicación {}
ID de teatro
TIPO DE GRÁFICO
Mapa de calor geográfic
Codificar
Filtrar
Personalizar
Coordenadas
geo
Intensidad
A _identificación
Agregar
CONTAR
AVANCE
COMPLETO
SUBCONJUNTO
Ubicación geográfica de los cines
Enter a description
Edmonton
Regina
Winnipeg
Seattle
Quebec
SAINT PIERRE AND MIQUELON
Salem
Boise
Toronto
Denver
UNITED STATES
Indianapolis
San Francisco
Los Angeles
Oklahoma City
Atlanta
BERMUDA
Juárez
Gulf of Mexico
MEXICO
BAHAMAS
CUBA
Honolulu
Mexico
JAMAICA
Folleto | © 1987-2026 HERE | Condiciones de uso
count (_id)
150
100
50